ID Share

Consent-based sharing of government ID documents in healthcare on Ethereum

BCS3210 Blockchains, Group 21
Sam Prior, Gyorgy Balint, Luka Toplak, Jack Bikar

The problem

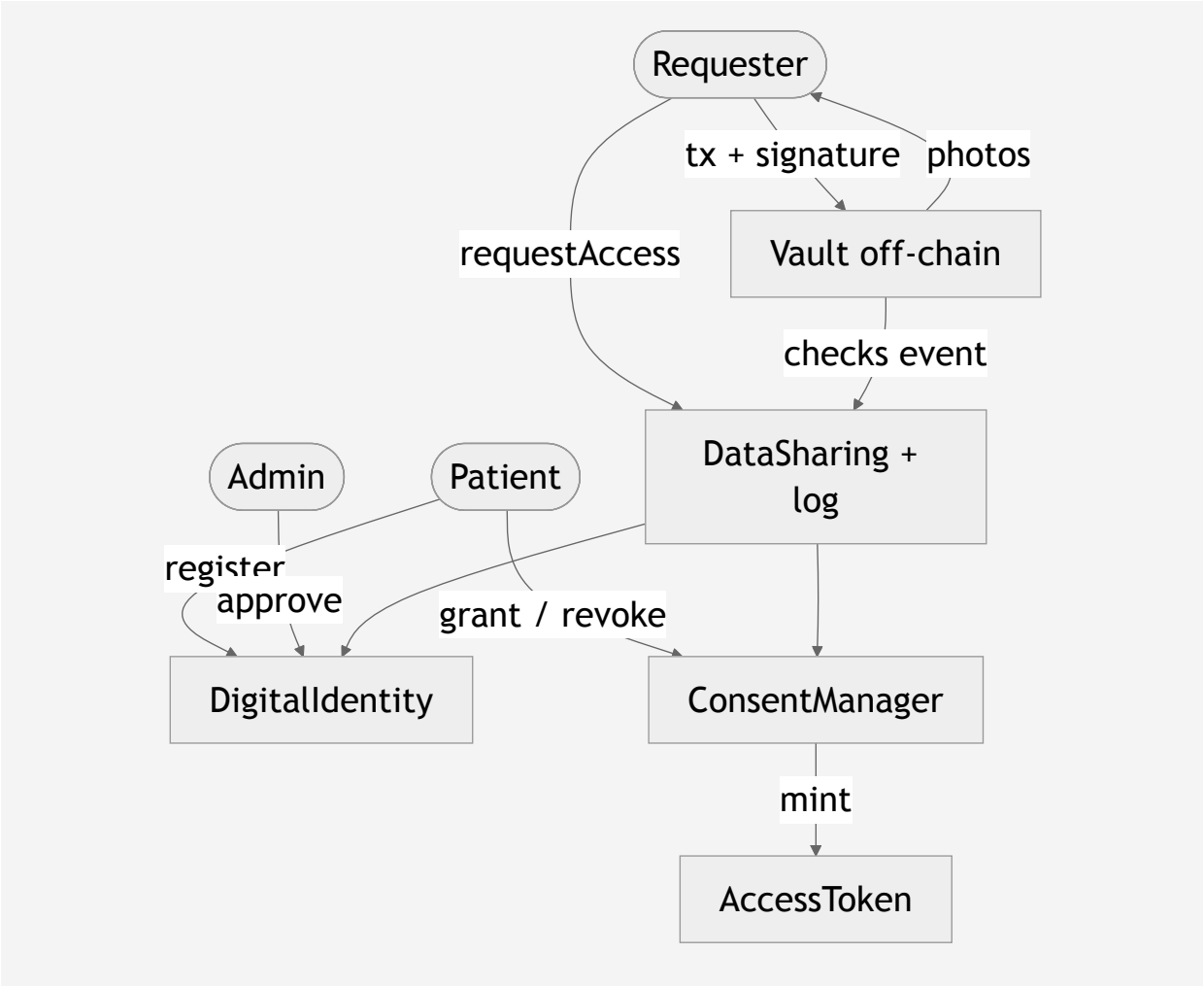
- Every clinic, lab and insurer keeps its own scan of your ID
- You don't know who has a copy or who looked at it
- You can't take a copy back
- No audit trail you can check yourself
- Every provider is a breach target: Change Healthcare 2024, **192.7 million** people affected
- Patients get nothing back for sharing

Our idea

- ID photos stay **with the patient** (off-chain vault)
- On-chain: only **hashes**, consents and the access log
- Consent: **1 to 365 days**, front / back / both, revocable any time
- **Every** access attempt logged, also the failed ones
- 10 ACT reward token for the first consent to a provider

Role	Can
Patient	register, grant / revoke consent, read log
Requester	request access, get allowed photos
Admin	approve requesters, pause

Architecture



- 4 contracts, one job each, talk through interfaces (like Lab 4)
- Only ConsentManager can mint tokens
- Python vault hands out the files
- Hardhat 3, Solidity 0.8.28, viem, web3.py

Design decision 1: failed requests must not revert

A `revert` also rolls back the log entry, so a denied attempt would leave no trace.

`requestAccess` therefore always writes the log and emits an event, and returns `true` or `false`.

```
if (!approved)  reason = NotApproved;
else if (!exists)  reason = NoConsent;
else if (revoked)  reason = Revoked;
else if (now >= expiresAt) reason = Expired;

granted = (reason == None);
accessLogs[patient].push(...);
emit granted ? AccessGranted
             : AccessDenied(reason);
```

Design decision 2: the chain is public

- Our first version returned the file link from the contract, but anyone can read contract storage
- So the **vault** enforces access, based on on-chain proof:

1. Requester calls `requestAccess`
2. Sends tx hash + signature to the vault
3. Vault checks: AccessGranted event, signer, age, not reused, consent still valid
4. Vault sends only the allowed photos
5. Requester checks SHA-256 against the chain

Every download has a matching entry in the on-chain audit log.

Testing

65 / 65 passing

Solidity tests with forge-std, incl. a fuzz test (256 runs), plus 11 Python tests for the vault

Only generated addresses and fake hashes, no personal data.

File	Tests
AccessToken.t.sol	11
DigitalIdentity.t.sol	16
ConsentManager.t.sol	21
DataSharing.t.sol	13
Integration.t.sol	4

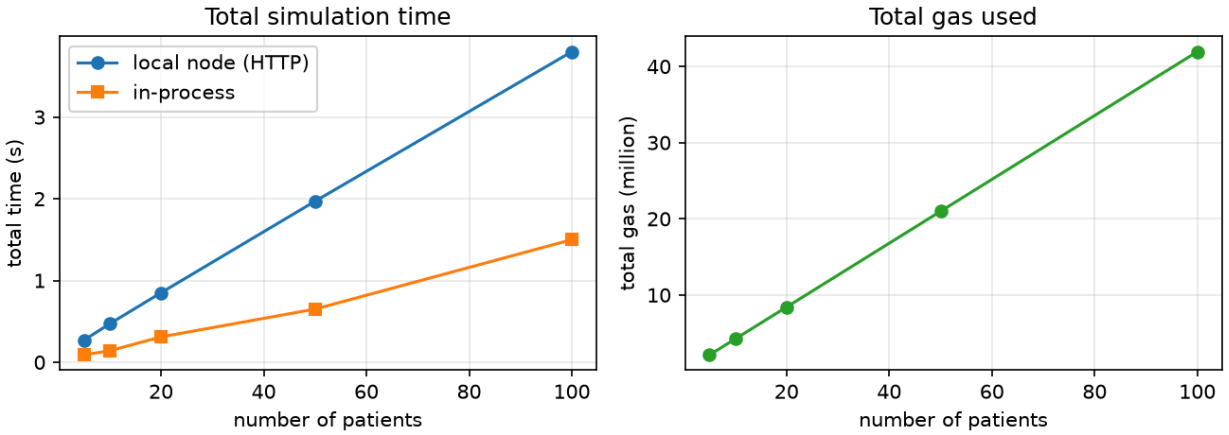
- Boundaries (0 / 1 / 365 / 366 days), exact expiry, reward only once, denied = logged not reverted

Gas optimisation

- Timestamps as `uint64`, so a consent and a log entry each fit in **one** storage slot
- Reward flag inside the packed consent
- `immutable` contract references
- Removed a redundant counter

Operation	Before	After
requestAccess (granted)	107,429	**70,108** (-35%)
grantConsent (first)	187,290	**94,835** (-49%)
registerUser	167,975	**145,921** (-13%)
Deployment (all)	3.14 M	3.19 M (+1.4%)

Simulation: 5 to 100 patients



- Gas per operation constant: no loops over users
- ~420k gas per patient lifecycle
- 100 patients: 470 tx in 3.8 s
- About 4 to 13 ms per operation on the local node
- 200 audit events read in 27 ms

Limitations and next steps

- Local chain: on mainnet ~12 s per block and real fees, so move to a Layer-2 rollup
- Access pattern (who asked whom, when) is public: zero-knowledge proofs could hide it
- Admin is one trusted party: use a multi-sig
- A requester can keep copies; the chain proves access but cannot force deletion
- Encrypt files for the requester, use MetaMask instead of test keys

Thank you

Questions?

Demo: `npm run demo`